

IPTU GERADOR

Documentação Técnica Completa — Versão 3.0

Autor: Bernardo Mozelli de Medeiros

1. Visão Geral do Sistema

O IPTU Gerador é uma aplicação web containerizada para a geração automatizada de carnês de IPTU (Imposto Predial e Territorial Urbano) a partir de ficheiros posicionais (.txt) exportados pelo ERP Supernova, sistema oficial de gestão tributária da Prefeitura Municipal de Santa Luzia/MG.

O sistema recebe os arquivos de dados, processa cada registro posicional de largura fixa, aplica o layout oficial do carnê e gera PDFs em lote prontos para impressão gráfica, com barra de código compatível, frente e verso e imagem institucional customizável.

1.1 Funcionalidades Principais

- Upload de múltiplos arquivos .txt via drag-and-drop ou seleção nativa
- Processamento em lote com geração paralela de PDFs por arquivo
- Barra de progresso e log de terminal em tempo real via polling
- Geração automática de nomes com timestamp (iptu-DD-MM-AAAA-HH-MM-nomeoriginal.pdf)
- Suporte a arquivos grandes: divisão automática em partes de N registros
- Imagem do verso customizável por sessão (substitui o padrão institucional)
- Cancelamento seguro com sinal de threading para o motor de processamento
- Download direto dos PDFs gerados pelo navegador
- Histórico persistente de sessões em banco SQLite
- Interface responsiva com paleta institucional: azul #0052A3, verde #7BC043, laranja #F39200

1.2 Stack Técnica

Camada	Tecnologia	Versão	Descrição
Frontend	React + Vite	18 / 5	Interface SPA containerizada
API REST	Python FastAPI	3.12	Endpoints de geração, progresso e histórico
Servidor API	Uvicorn	0.29+	ASGI server para FastAPI
Geração PDF	ReportLab	4.0+	Motor de renderização do carnê
Imagens	Pillow	10.0+	Validação e manipulação de imagens
Banco de Dados	SQLite	—	Histórico de sessões e logs (arquivo local)
Proxy / Serve	nginx (Alpine)	—	Serve o build React e proxia /api
Containers	Docker Compose	3.9	Orquestração de dois serviços

1.3 Arquitetura Geral

O sistema opera com dois containers Docker em rede interna (iptu_net):

- backend (Python/FastAPI): API REST na porta 8000. Recebe os arquivos, executa o motor de geração em thread separada, serve os PDFs gerados e o histórico SQLite.
- frontend (nginx + React build): Serve a SPA React na porta 80. Proxía /api/* e /pdfs/* para o backend via nginx, eliminando CORS e expondo apenas uma porta ao exterior.

O volume Docker iptu_data monta /app/data no backend, garantindo persistência dos PDFs gerados e do banco SQLite entre reinicializações dos containers.

2. Estrutura do Projeto

```
iptu-docker/
├── backend/
│   ├── Dockerfile
│   ├── main.py           ← API FastAPI (nova camada)
│   ├── motor_iptu.py     ← Motor de geração PDF (original, intacto)
│   ├── db.py             ← Persistência SQLite (original, intacto)
│   ├── requirements.txt
│   └── static/img/
│       ├── logo_prefeitura.png
│       ├── logo.png
│       ├── iptu-verso.jpg ← Verso padrão do carnê
│       └── favicon.ico
├── frontend/
│   ├── Dockerfile        ← Build Vite → nginx
│   ├── nginx.conf        ← Proxy /api e /pdfs → backend
│   ├── index.html        ← Entry point Vite
│   ├── vite.config.js
│   ├── package.json
│   └── src/
│       ├── main.jsx
│       ├── App.jsx       ← Componente raiz e orquestrador
│       └── components/
│           ├── Header.jsx
│           ├── Card.jsx
│           ├── FileDropzone.jsx
│           ├── ProgressPanel.jsx
│           ├── ResultsPanel.jsx
│           └── HistoricoPanel.jsx
│       └── utils/
│           └── api.js     ← Client Axios para a API
├── docker-compose.yml
└── README.md
```

2.1 Backend (Python/FastAPI)

Arquivo	Descrição
main.py	Ponto de entrada da API FastAPI. Define todos os endpoints REST, gerencia o estado dos jobs em memória (progresso, logs, PDFs gerados), inicia threads de geração e faz interface com motor_iptu.py e db.py.
motor_iptu.py	Motor original de geração de PDF. Lê arquivos posicionais .txt em modo binário (encoding latin-1 por posição de byte), renderiza frente e verso do carnê usando ReportLab com layout completo de parcelas, barcodes, logotipos e dados do contribuinte. Intacto — sem modificações.
db.py	Módulo de persistência SQLite. Gerencia sessões de geração, logs linha a linha e registro de PDFs produzidos. Thread-safe via threading.Lock e WAL mode. Intacto — sem modificações.
requirements.txt	Dependências Python: fastapi, uvicorn[standard], python-multipart, reportlab, Pillow.
Dockerfile	Imagem python:3.12-slim. Instala dependências de sistema (libfreetype, libjpeg, zlib) para ReportLab e Pillow, depois instala as dependências Python e copia o código.
static/img/	Imagens institucionais servidas via /static/. Inclui logos e a imagem padrão do verso do carnê (iptu-verso.jpg).

2.2 Frontend (React/Vite)

Arquivo / Pasta	Descrição
App.jsx	Componente raiz. Orquestra o fluxo completo: coleta de arquivos → envio multipart → polling de progresso → exibição de resultados. Gerencia o estado do job ativo (jobId, progresso, status) e inicia/encerra o polling a cada 1,2 segundos.
components/Header.jsx	Cabeçalho fixo com barra tricolor (azul/verde/laranja), logos da prefeitura e badge com o ano do exercício atual (calculado no cliente).
components/Card.jsx	Componente de card reutilizável com CardHeader (ícone, título, subtítulo, trailing action). Base visual de todos os painéis da aplicação.
components/FileDropzone.jsx	Área de drag-and-drop baseada em react-dropzone. Suporta múltiplos arquivos .txt ou imagem única (verso). Exibe lista de arquivos selecionados com nome, tamanho e botão de remoção individual.
components/ProgressPanel.jsx	Painel de progresso em tempo real. Exibe barra de progresso percentual, log de terminal scrollável (fundo escuro #1A2233, estilo de console), botão de cancelar e indicador de status com cor por estado (azul/verde/laranja/vermelho).
components/ResultsPanel.jsx	Lista de PDFs gerados após conclusão. Cada item mostra nome, tamanho em MB, botão de download direto (âncora HTML5) e botão

Arquivo / Pasta	Descrição
	de exclusão com chamada à API.
components/ HistoricoPanel.jsx	Painel de histórico de sessões. Busca dados via GET /api/historico e GET /api/estatisticas. Exibe badges coloridos por status, nome dos arquivos e data de início. Recarregável manualmente.
utils/api.js	Client Axios pré-configurado com baseURL /api. Exporta funções tipadas: gerarIPTU (multipart), getProgresso, cancelarJob, excluirPDF, getHistorico, getDetalhe, getStats.
vite.config.js	Configuração do Vite. Define proxy de desenvolvimento (/api, /pdfs, /static → backend:8000) e diretório de output dist/.
nginx.conf	Configuração nginx do container frontend. Serve os arquivos estáticos do build Vite, proxia /api/ e /pdfs/ para o backend, e redireciona tudo mais para index.html (SPA routing).

3. Banco de Dados (SQLite)

O banco SQLite é criado automaticamente em data/logs/iptu.db dentro do volume Docker iptu_data na primeira execução. As tabelas são inicializadas pela função `_criar_tabelas()` em db.py.

3.1 Tabelas

Tabela	Descrição	Principais Colunas
sessoes	Registro de cada sessão de geração. Uma sessão = um clique em Gerar PDFs.	id, iniciado_em, finalizado_em, status, arquivos_entrada, total_registros, total_paginas, num_pdfs, pasta_saida, erro_msg
logs	Log linha a linha de cada sessão. Alimentado pelo motor via log_fn callback.	id, sessao_id, momento, tipo (info/ok/warn/err), mensagem
pdfs_gerados	Registro dos arquivos PDF produzidos. Permite controle de exclusão.	id, sessao_id, nome, caminho, tamanho_mb, criado_em, excluido, excluido_em

3.2 Status de Sessão

Status	Descrição
em_andamento	Sessão iniciada, motor rodando.
concluido	Todos os arquivos processados com sucesso.
cancelado	Interrupção solicitada pelo usuário via botão Cancelar.
erro	Exceção não tratada durante o processamento.

4. API REST (porta 8000)

Todos os endpoints são servidos pelo FastAPI no container backend. Em produção, o acesso externo passa pelo nginx do frontend na porta 80, que proxia `/api/*` → `backend:8000`.

4.1 Endpoints

Rota	Método	Descrição
/api/health	GET	Health check. Retorna status ok e o ano atual.
/api/gerar	POST	Recebe multipart/form-data com os arquivos .txt (campo arquivos) e imagem de verso opcional (campo verso). Retorna { job_id } imediatamente e inicia a geração em background.
/api/progresso/{job_id}	GET	Retorna o estado atual do job: status, percent (0–100), log (lista de strings), pdfs (lista de objetos com nome, url, tamanho_mb).
/api/cancelar/{job_id}	POST	Sinaliza cancelamento via threading.Event. O motor verifica o sinal a cada registro processado.
/api/pdf	DELETE	Exclui um PDF do disco. Parâmetro: caminho (query string). Registra exclusão no banco via <code>db.marcar_pdf_excluido()</code> .
/api/historico	GET	Lista sessões do banco SQLite. Parâmetro opcional: limite (padrão 30).
/api/historico/{sessao_id}	GET	Detalhe de uma sessão específica com logs e lista de PDFs.
/api/estatisticas	GET	Estatísticas globais: total de sessões, concluídas, canceladas, com erro, registros e PDFs processados.
/api/verso-padrao	GET	Retorna a URL da imagem de verso padrão (<code>/static/img/iptu-verso.jpg</code>).
/pdfs/{job_id}/{arquivo}	GET	Serve os PDFs gerados diretamente (StaticFiles do FastAPI).
/static/{arquivo}	GET	Serve arquivos estáticos do backend (logos, imagens).

4.2 Fluxo de Geração

Etapa	Responsável	O que acontece
1. Upload	App.jsx + api.js	Usuário seleciona arquivos. FormData enviado via POST /api/gerar com multipart.
2. Recebimento	main.py	Arquivos salvos em data/uploads/{job_id}/. threading.Event de cancelamento criado. Worker thread iniciada. job_id retornado.
3. Processamento	motor_iptu.py	Worker chama processar_arquivo() para cada .txt. Motor lê arquivo em modo binário, renderiza frente+verso por registro, grava PDF em data/pdfs/{job_id}/.
4. Progresso	main.py + ProgressPanel.jsx	Frontend faz polling GET /api/progresso/{job_id} a cada 1,2s. Exibe log e barra de progresso.
5. Conclusão	main.py + ResultsPanel.jsx	Motor retorna _Resultado com .total e .pdfs. Status atualizado para concluído. Frontend exibe links de download.
6. Cancelamento	main.py + ProgressPanel.jsx	POST /api/cancelar/{job_id} seta o Event. Motor verifica a cada registro. PDFs parciais preservados.

5. Motor de Geração de PDF (motor_iptu.py)

O motor_iptu.py é o núcleo original do sistema, preservado integralmente da versão Flet desktop. Processa arquivos posicionais de largura fixa exportados pelo ERP Supernova.

5.1 Layout Posicional

Cada linha do arquivo .txt representa um registro IPTU com campos em posições fixas de bytes. A leitura é feita em modo binário (rb) para garantir posicionamento correto de caracteres acentuados latin-1 que deslocariam posições se lidos como UTF-8.

Campo	Posição Inicial	Posição Final	Descrição
NomeDestinatario	1	80	Nome do destinatário para envio postal
EnderecoCorres	81	160	Endereço de correspondência
NomeContribuinte	238	317	Nome do contribuinte
EnderecoContrib	318	397	Endereço do contribuinte
Exercicio	794	797	Ano do exercício financeiro
InscricaoCadastral	798	827	Inscrição cadastral do imóvel
ValorVenal	930	949	Valor venal do imóvel
TotalaPagarCDesconto	1186	1205	Valor total a pagar com desconto
Parcelas (7 parcelas)	1476–2927	—	Blocos de 244 bytes cada para cota única + 6 parcelas

5.2 Assinatura da Função Principal

```
def processar_arquivo(
    arquivo_dados,      # str: caminho do .txt de entrada
    arquivo_saida,      # str: caminho base do .pdf de saída
    imagem_propaganda,  # str: caminho da imagem do verso
    encoding='utf-8',   # ignorado (leitura binária)
    logo_path=None,     # str opcional: caminho do logo
    registros_por_parte=N, # int: máx. registros por PDF (split automático)
    cancelar_flag=None,  # threading.Event: set() cancela
    log_fn=None,        # callable(msg, tipo): log em tempo real
) -> _Resultado        # herda de int (total) + .pdfs (lista de caminhos)
```

5.3 Divisão Automática em Partes

Para arquivos com mais de registros_por_parte registros, o motor divide automaticamente a saída em múltiplos PDFs numerados:

```
iptu-10-05-2026-1200-contribuintes_parte_01.pdf
iptu-10-05-2026-1200-contribuintes_parte_02.pdf
...
```

O split ocorre sem carregar todo o arquivo em memória — o motor usa streaming linha a linha, mantendo no máximo um canvas ReportLab ativo por vez.

6. Interface do Usuário

6.1 Fluxo de Uso

Passo	Ação do Usuário	Resposta do Sistema
1	Abre http://[servidor]	Interface carregada. Histórico buscado automaticamente.
2	Arrasta arquivos .txt para a área de upload	Arquivos listados com nome e tamanho. Botão Gerar PDFs ativado.
3	(Opcional) Substitui imagem do verso	Preview da nova imagem exibido ao lado do padrão atual.
4	Clica em Gerar PDFs	Upload dos arquivos. Painel de progresso aparece com log em tempo real.
5	Aguarda processamento	Barra avança. Log mostra cada registro processado. Botão Cancelar disponível.
6	Processamento concluído	Painel de resultados aparece com lista de PDFs e botões de download.
7	Clica em Baixar em um PDF	Download direto via link HTML5. Nenhuma lógica adicional no servidor.
8	Consulta histórico	Painel de histórico exibe sessões anteriores com status e estatísticas globais.

6.2 Paleta Visual

Cor	Hex	Uso
Azul institucional	#0052A3	Cabeçalho, botão primário, ícones, barra de progresso
Verde	#7BC043	Ícone verso, status concluído, badges
Laranja	#F39200	Status cancelado, alertas
Vermelho	#C0392B	Erros, botão excluir, status erro
Fundo geral	#F0F2F5	Background da página
Card fundo	#FFFFFF	Fundo dos painéis
Log terminal	#1A2233	Fundo do painel de log estilo console

7. Infraestrutura Docker

7.1 docker-compose.yml — Serviços

Serviço	Imagem Base	Porta Externa	Função
backend	python:3.12-slim	— (interna)	API FastAPI, motor PDF, SQLite
frontend	nginx:alpine (build node:20-alpine)	80:80	Serve React + proxy para backend

7.2 Volumes e Rede

Recurso	Tipo	Descrição
iptu_data	Volume nomeado	Persistência de data/pdfs/, data/uploads/, data/logs/iptu.db
iptu_net	Rede bridge	Rede interna entre frontend e backend. Backend não expõe porta ao host.

7.3 Build do Frontend

O Dockerfile do frontend usa multi-stage build:

- Stage 1 (node:20-alpine): instala dependências npm, executa npm run build (Vite), gera dist/.
- Stage 2 (nginx:alpine): copia dist/ para /usr/share/nginx/html e aplica nginx.conf com proxy.

O stage 1 não entra na imagem final, mantendo-a pequena (~25MB).

8. Guia de Desenvolvimento

8.1 Alterar o Layout do Carnê

O layout visual do carnê é definido exclusivamente em `motor_iptu.py`. Principais pontos de modificação:

- `LAYOUT` (dict) — mapeamento de campos para posições no arquivo `.txt`
- `PARCELAS_LAYOUT` (list) — posições das 7 parcelas (cota única + 6 mensais)
- `desenhar_frente(canvas, dados, ...)` — renderização da frente do carnê
- `desenhar_verso(canvas, dados, imagem, ...)` — renderização do verso com imagem

8.2 Adicionar Novo Endpoint à API

- Criar a função no `main.py` com o decorador `@app.get`, `@app.post`, etc.
- Adicionar a função correspondente em `frontend/src/utils/api.js`
- Usar o novo endpoint nos componentes React via import de `api.js`

8.3 Adicionar Novo Componente React

- Criar o arquivo em `frontend/src/components/NomeComponente.jsx`
- Importar e usar em `App.jsx` ou no componente pai
- Para rebuild em produção, reconstruir o container frontend: `docker compose up --build frontend`

8.4 Comandos Úteis

Comando	Descrição
<code>docker compose up --build -d</code>	Sobe e reconstrói todos os containers
<code>docker compose restart backend</code>	Reinicia o backend após alteração de <code>main.py</code>
<code>docker compose logs -f backend</code>	Ver logs da API Python em tempo real
<code>docker compose logs -f frontend</code>	Ver logs do nginx em tempo real
<code>docker exec -it iptu_backend bash</code>	Acessar o shell do container backend
<code>docker compose down</code>	Para os containers (dados preservados no volume)
<code>docker compose down -v</code>	Para e apaga os containers e o volume (dados perdidos)

9. Dependências

9.1 Backend (Python)

Pacote	Versão Mínima	Função
fastapi	0.110.0	Framework web assíncrono para a API REST
uvicorn[standard]	0.29.0	Servidor ASGI (inclui websockets e httptools)
python-multipart	0.0.9	Parsing de multipart/form-data para upload de arquivos
reportlab	4.0.0	Geração de PDFs com layout completo
Pillow	10.0.0	Validação e abertura de imagens para o verso

9.2 Frontend (Node.js)

Pacote	Versão	Função
react	^18.3.1	Biblioteca UI principal
react-dom	^18.3.1	Renderização no DOM
axios	^1.7.2	Cliente HTTP para chamadas à API
react-dropzone	^14.2.3	Componente de drag-and-drop de arquivos
react-hot-toast	^2.4.1	Notificações toast não-intrusivas
lucide-react	^0.383.0	Biblioteca de ícones SVG
vite	^5.4.0	Bundler e dev server (substituto do CRA)
@vitejs/plugin-react	^4.3.1	Plugin React para Vite (JSX transform, HMR)

10. Arquivos Originais Preservados

Os seguintes arquivos foram migrados da versão desktop Flet sem qualquer modificação de lógica de negócio:

Arquivo	Origem	Motivo da Preservação
motor_iptu.py	app_iptu_flet/app/	Núcleo do sistema. Layout, parsing posicional, geração ReportLab e toda a lógica de renderização do carnê. Testado e aprovado para produção.
db.py	app_iptu_flet/app/	Esquema SQLite, migrations e toda a API de persistência já validados em campo. Nenhuma mudança de estrutura necessária.